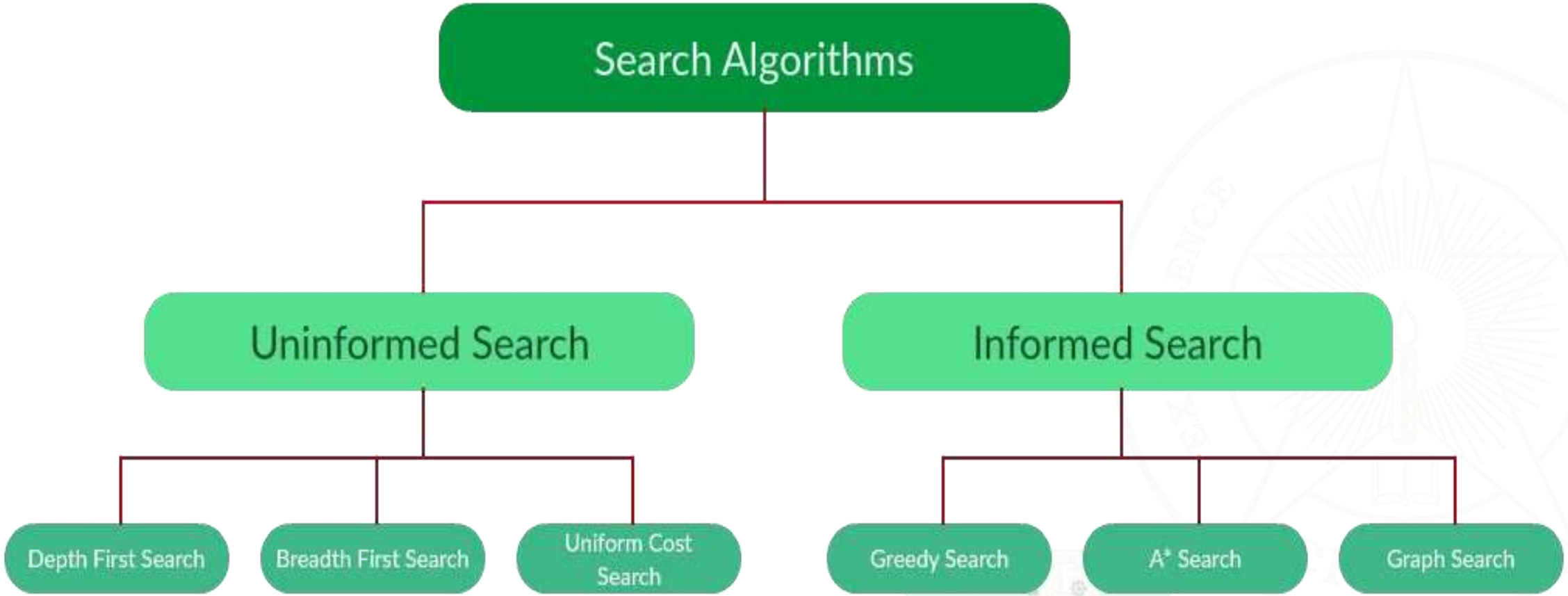
UNIT 02 - Part 1

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed Search	Informed Search
1. Search is done without any extra information.	1. Search is done with the help of information.
2. It has no knowledge about the goal direction.	2. It uses knowledge or heuristic value to find the solution.
3. It is more time-consuming.	3. It gives a quicker solution.
4. It has more time and space complexity.	4. It has less time and space complexity.
5. Examples: DFS, BFS, Uniform Cost Search.	5. Examples: A*, Best First Search.

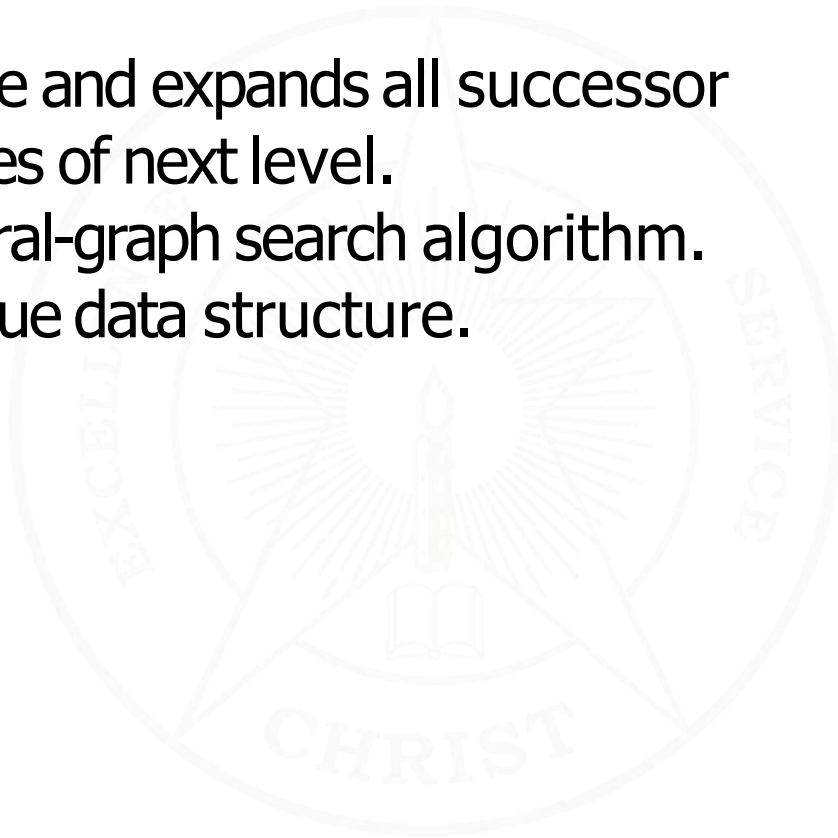
1. Breadth-first Search

Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.

BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.

The breadth-first search algorithm is an example of a general-graph search algorithm.

Breadth-first search implemented using FIFO queue data structure.



Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

Disadvantages:

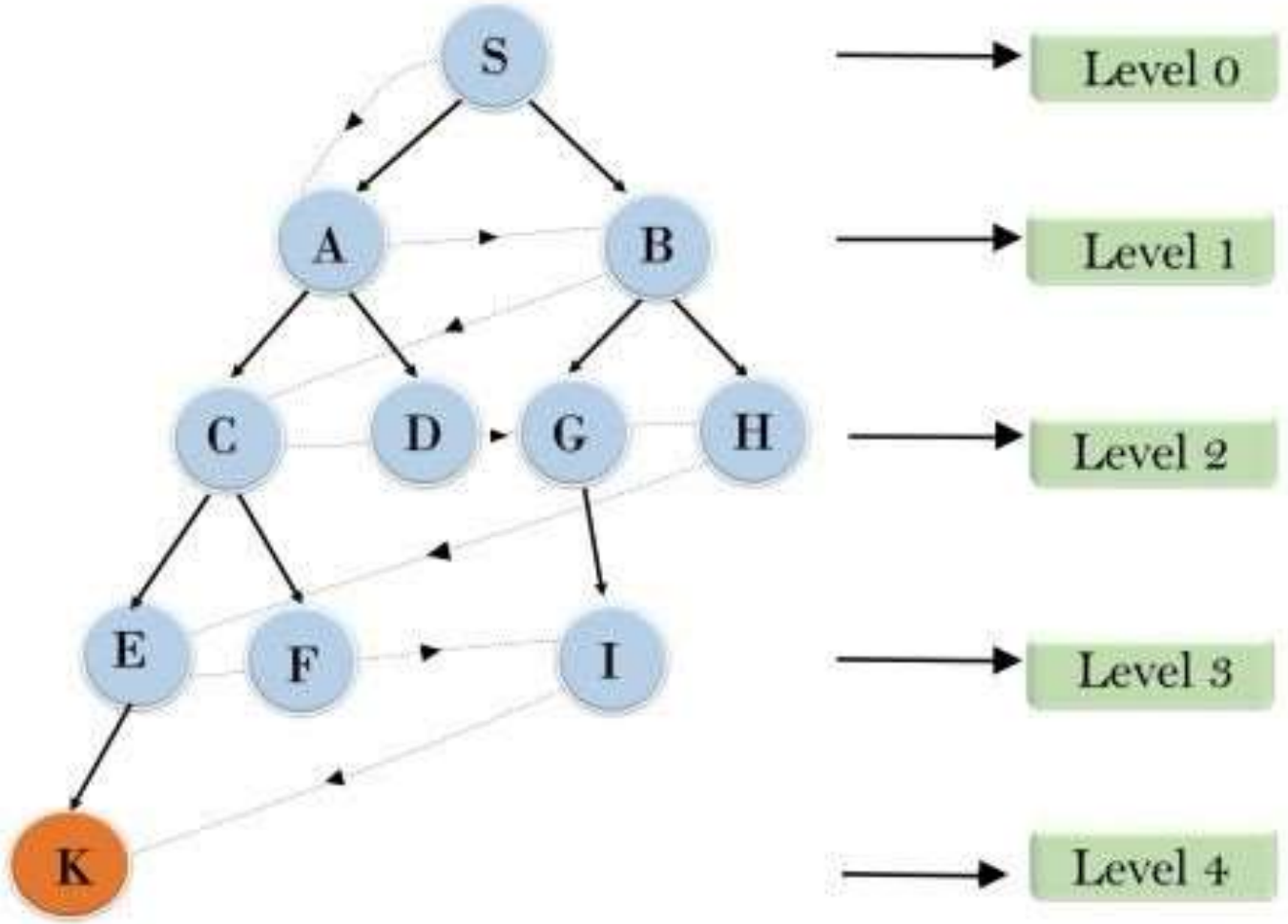
- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

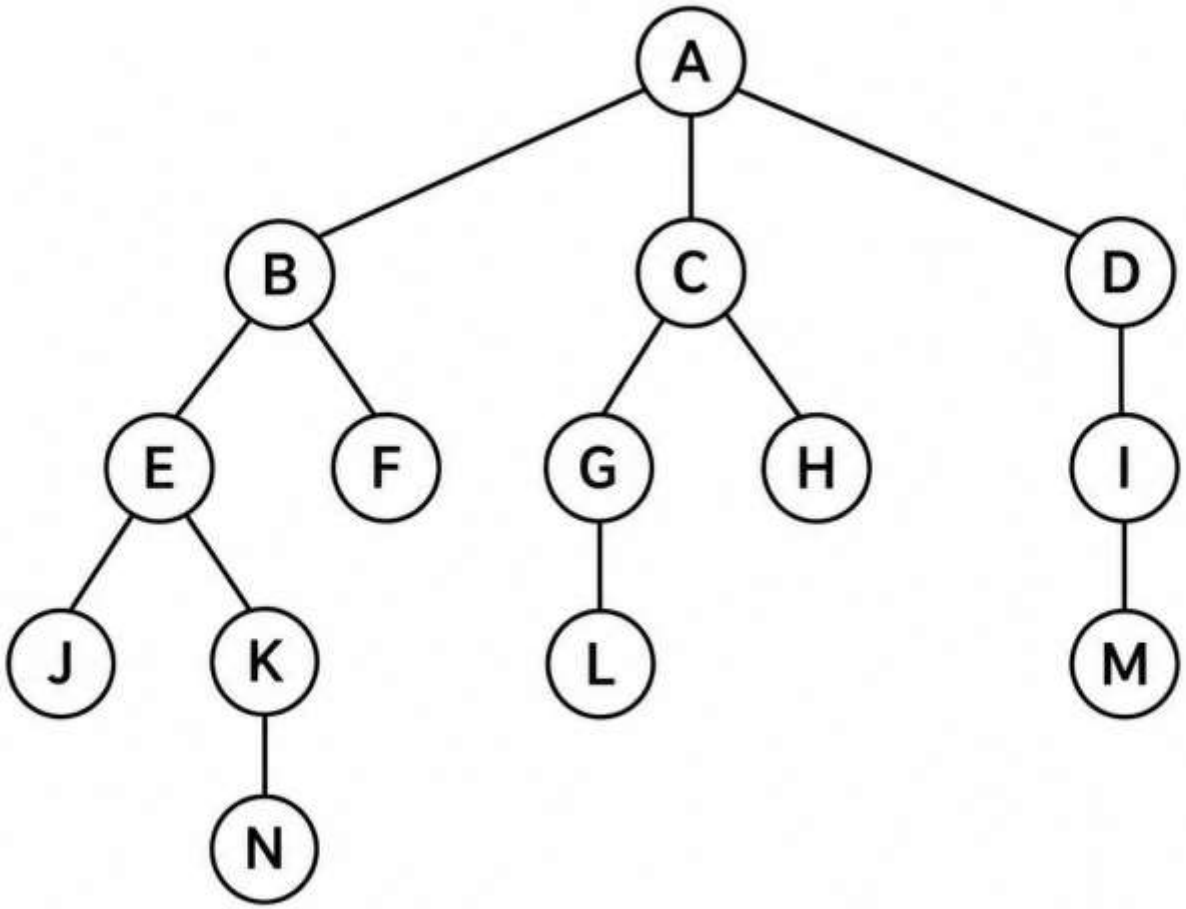
In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K

Breadth First Search

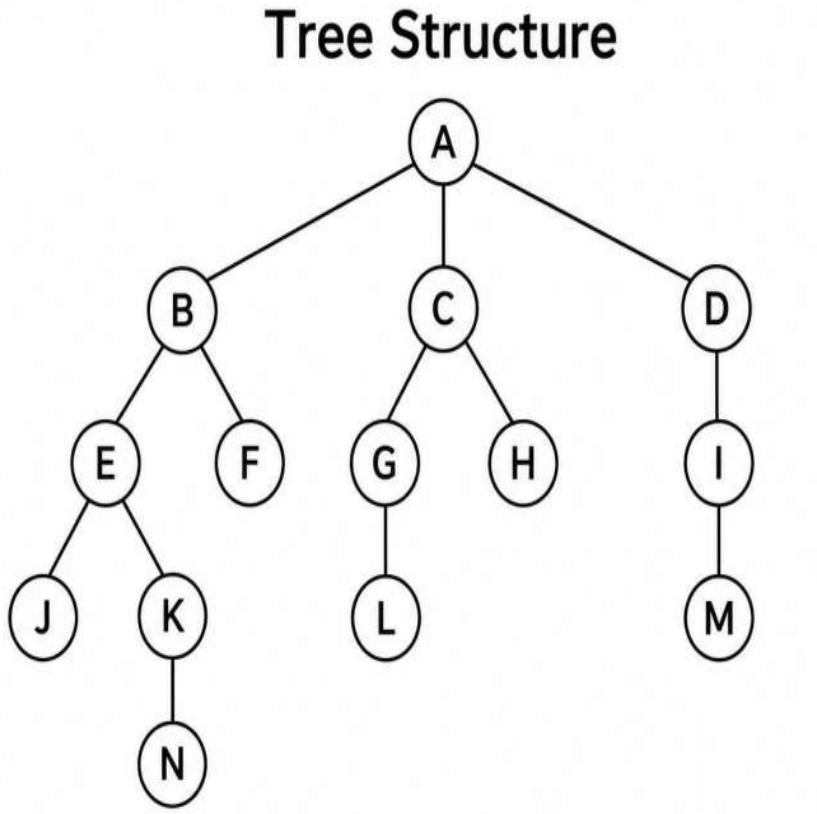


Tree Structure



Level 0: A | Level 1: B, C, D | Level 2: E, F, G, H, I | Level 3: J, K, L, M | Level 4: N

Step	Node Removed / Visited	Nodes Added	Queue Status
1	—	A	[A]
2	A	B, C, D	[B, C, D]
3	B	E, F	[C, D, E, F]
4	C	G, H	[D, E, F, G, H]
5	D	I	[E, F, G, H, I]
6	E	J, K	[F, G, H, I, J, K]
7	F	—	[G, H, I, J, K]
8	G	L	[H, I, J, K, L]
9	H	—	[I, J, K, L]
10	I	M	[J, K, L, M]
11	J	—	[K, L, M]
12	K	N	[L, M, N]
13	L	—	[M, N]
14	M	—	[N]
15	N	—	[ ]



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$$T(b) = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.
-

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantages

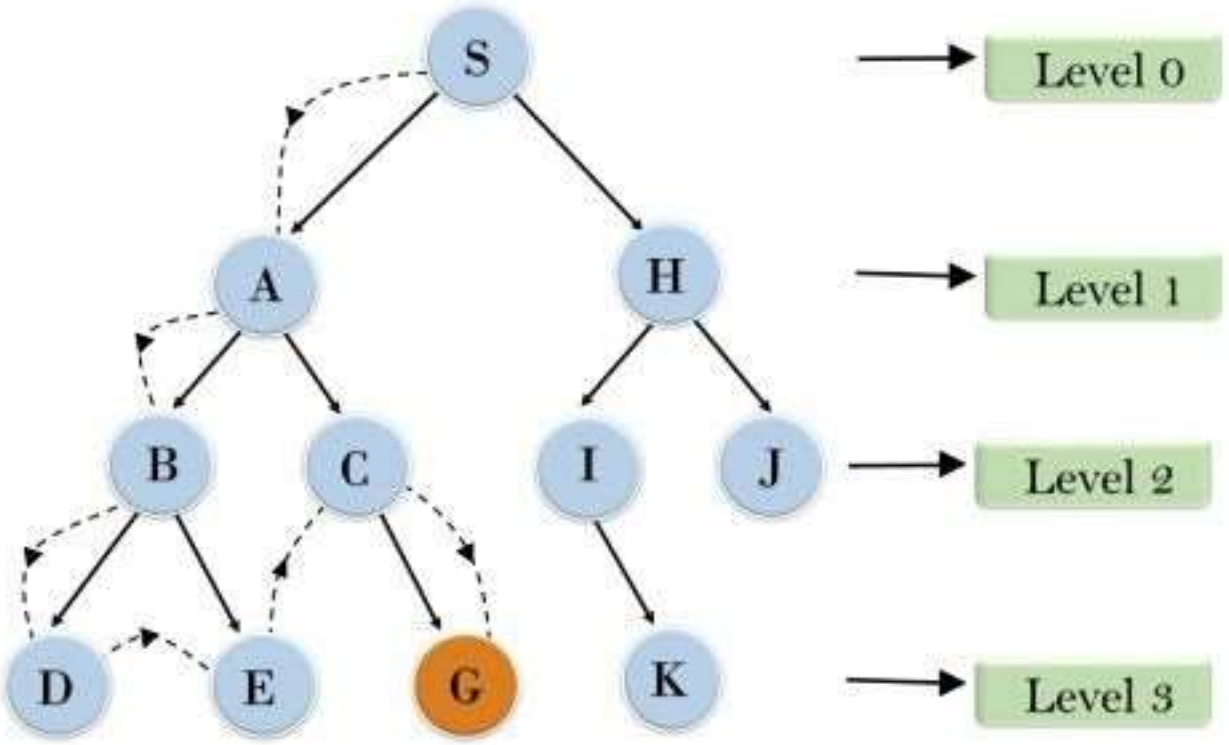
- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

Example:

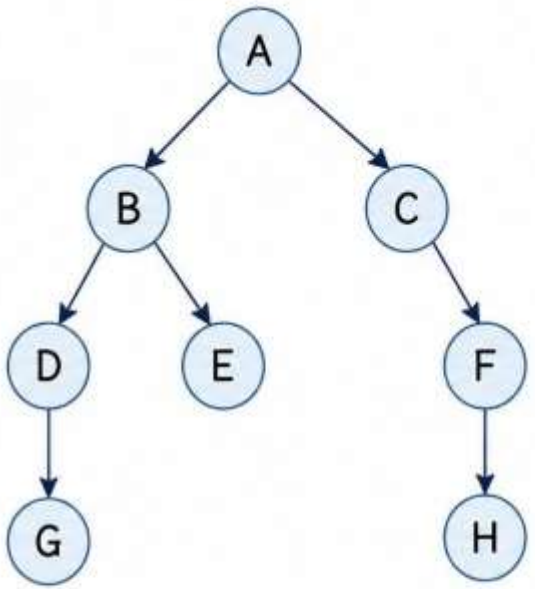
In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Depth First Search



Tree



DFS Traversal Order

A → B → D → G → E → C → F → H

Legend:

Top of Stack

Stack Representation (DFS using Stack)

Step	Operation	Stack (Top → Left)
1	Push start node A	A
2	Pop A, push C then B	B C
3	Pop B, push E then D	D E C
4	Pop D, push G	G E C
5	Pop G	E C
6	Pop E	C
7	Pop C, push F	F
8	Pop F, push H	H
9	Pop H	Empty

Traversal Order (DFS):

A → B → D → G → E → C → F → H

Completeness: DFS is complete only when the search space is finite or depth-limited.
Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:
$O(b^m)$
Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)
Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.
Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

Iterative Deepening Search

Iterative Deepening Search is a search technique that combines the advantages of **Breadth-First Search** and **Depth-First Search**.

It is also called **Iterative Deepening Depth-First Search** or **IDDFS**.
In this method, DFS is applied repeatedly with increasing depth limits until the goal node is found.

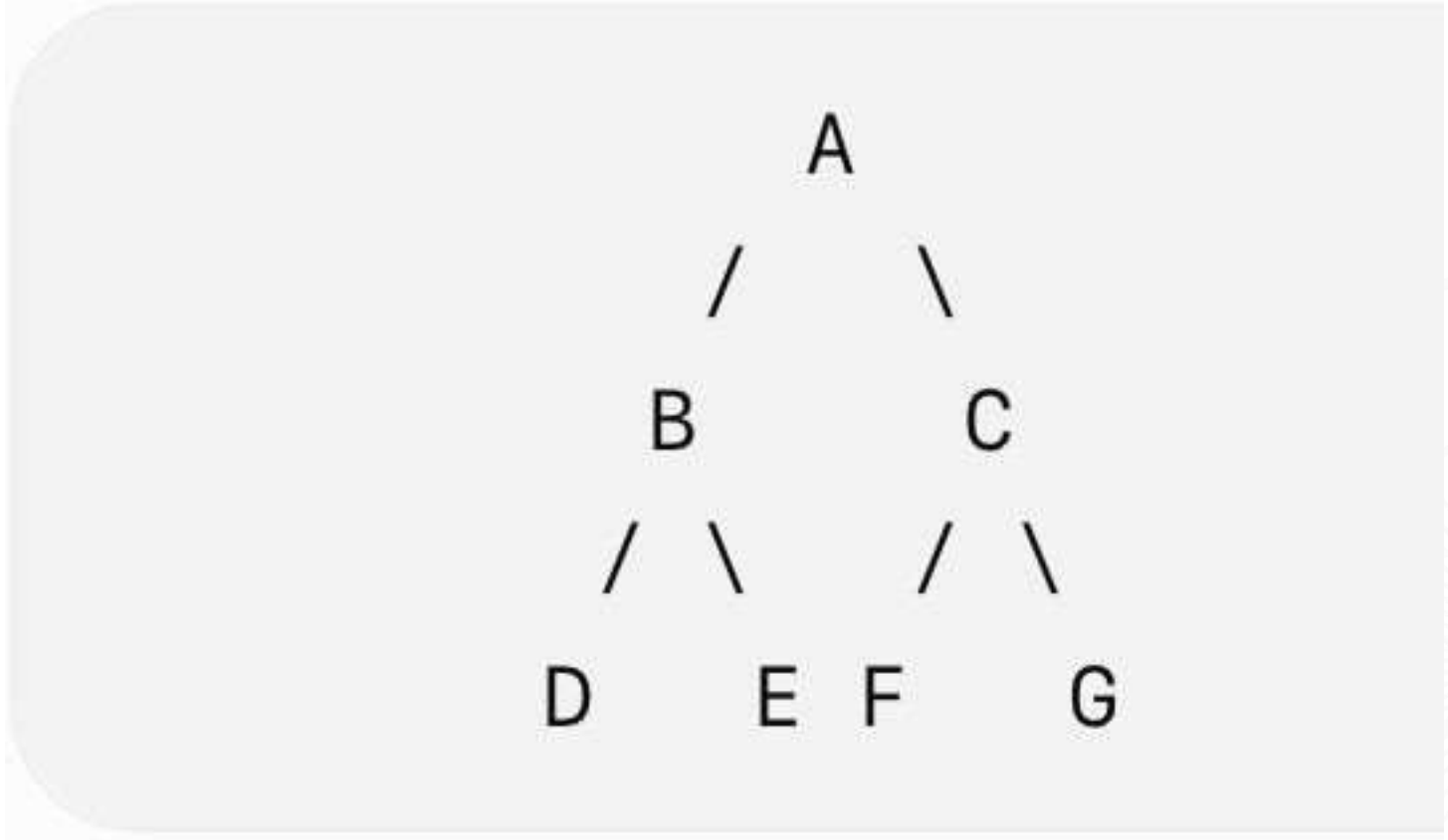
Basic Idea

In normal DFS, the search may go very deep in one path.
In Iterative Deepening, DFS is controlled using a **depth limit**.

The search is performed as:

Depth limit = 0
Depth limit = 1
Depth limit = 2
Depth limit = 3

The depth limit keeps increasing until the goal is reached.



Suppose the goal node is **G**.

Search Process

Iteration 1: Depth limit = 0

A

Goal not found.

Iteration 2: Depth limit = 1

A, B, C

Goal not found.

Iteration 3: Depth limit = 2

A, B, D, E, C, F, G

Goal found.

Final path:

A → C → G

Why Iterative Deepening is Used

- Iterative Deepening is useful because it avoids the disadvantages of DFS.
- DFS may go too deep in one branch and may miss a shallow solution for a long time.
- Iterative Deepening searches level by level like BFS, but it uses less memory like DFS.

Advantages

1. Complete if the branching factor is finite.
2. Optimal if all step costs are equal.
3. Uses less memory than BFS.
4. Finds the shallowest goal node.
5. Useful when the depth of the solution is not known.

Disadvantages

1. Some nodes are visited again and again.

2. It may take extra time due to repeated DFS calls.

3. It is not suitable when repeated computation is very costly.

Iterative Deepening Search is a search method where DFS is repeatedly applied with increasing depth limits.

It is useful because it combines the **completeness and optimality of BFS** with the **low memory requirement of DFS**.

Thank You